Where the screener tool keeps its data

Every database table and file a screener tool uses: what it holds, who writes it, who reads it, and whether it survives a restart, a deploy and a lost box. Written from `src/db/index.js` and the code that reads each file.

Related: the registers column by column (REGISTERS.md), screener definitions (SCREENER-BUILDER.md), settings (SETTINGS.md).

The rule

Each tool owns its database, and everything else reaches it **over HTTP, never by opening it**: qp reads the registers through `/api/warehouse/*`. The one exception is the archive, which opens a stopped tool's database read-only. Everything under `data/` is outside git, so a deploy never touches it.

---

1. A screener tool's database

`data/tradedesk.db` for T1, `data/t<n>.db` for the others (`DB_PATH`). SQLite in WAL mode. One per tool, same schema.

Table	Key	Holds	Written by	Read by
`screeners`	`id`, unique `key`	each screener: name, `filters` (JSON), `sort`, `limit_n`, run window `run_from` / `run_to`, check window `check_from` / `check_to`, `mirror_of`, `label_only`, `paused_at` / `paused_reason`, `enabled`	the screener builder; seeded at startup	every scan
`settings`	`key`	every setting (SETTINGS.md), plus `tool.name` and `tool.paused`	Settings tab, pause/rename	all stages
`scheduler_jobs`	`job_id`	each job's schedule and on/off	Monitor tab	the scheduler at startup
`r0_checkpoint`	`id = 1`	today's whole live list as JSON, saved after every full scan	each scan	startup: restored if it is from today
`r1_frozen`	`date, ticker`	**the photo**: the full card JSON at the capture minute	R1 capture	R4 join, card history, qp
`r2_market_snapshots`	`id`	eight market snapshots 09:26–10:01 (`slot`, JSON)	R2 jobs	export
`r3a`, `r3b`	`date, ticker`	outcome from entry A / B: entry price, high, low, ATR14, upR, downR, opening range high/low	R3 at 16:05	R4 join, scorecard
`r4a_train`, `r4b_train`	`date, ticker`	the training rows (R1 ⋈ R3) as JSON, with `source` (capture or upload)	R3 sync, CSV upload	scorer training
`shortlist`	`date`	that day's starred tickers (JSON) and whether exported	★ and the auto-rule	Shortlist tab; published to the unified shortlist
`analysis_model`	`id`	an older model record	nothing today	nothing (dead)

Migrations. `CREATE TABLE IF NOT EXISTS` does not add new columns to an old database, so `ensureColumns()` adds missing ones at startup (screener windows, check windows, `label_only`,

`mirror_of`, pause fields, opening range). This crashed T2 and T3 before it existed. A rebuild needs real, versioned migrations.

Read-only mode. With `DB_READONLY=1` (the archive), the database is opened read-only and nothing is created or migrated.

In memory only (lost on restart)

State	Effect of a restart
r0, today’s live list	restored from `r0_checkpoint` if it is from today; anything found since the last full scan is lost
hot-sector state (`sideD/sectors.js`)	every sector starts not hot; sustained counts restart
job run history (last run, status, error)	blank until each job runs again
scan status and last report	blank until the next scan

---

2. Shared files in `data/`

File	Holds	Written by	Read by
`auth.json`	the password hash and the cookie-signing secret	`scripts/set-password.sh`	every program's login
`keys.json` (0600)	the desk-wide API keys (Alpaca, Finnhub …)	by hand	every tool (re-read every 30 s), the journal's bar fetch
`tools.local.json`	tools added from the page and overrides	Manage tools	the registry, so every program
`tool-stages.json`	manual stage overrides per tool	`/api/tools/:id/stage`	`/api/tools`
`industry-map.json`	symbol → industry and group	qp (SIC, nightly) and tool scans (TradingView labels, never over a SIC entry)	tools, qp group ranks
`oneil-market.json`	O'Neil's market model	qp	the Market tab
`oneil-groups.json`	group ranks	qp	the Market tab, cards
`oneil-13f.json`	institutional holders (I)	qp	cards
`oneil/`	EDGAR company facts, `_tickers.json`	qp	C and A tables
`shortlist-all.json`	the union of every tool's shortlist	every tool, its own entry only	every tool (★ also … on cards), `/api/shortlist/all-tools`
`r2.json`	Cloudflare R2 backup credentials	by hand	the nightly backup

`tools.config.json` (in git) holds the built-in tools; `data/tools.local.json` is merged over it (TOOL-LIFECYCLE.md).

3. The scorer

Each tool's scorer writes its trained tables to `src/scoring/outputs` (T1) or `src/scoring/outputs-t<n>` (`MODEL_OUTPUT_ROOT`), and reads training CSVs the tool exports to its temp folder. See SCORING-TODAY.md.

What survives what

Event	Tool databases, `data/`	In-memory state
process restart	kept	lost (r0 restored from checkpoint)
deploy (`git pull`)	kept (outside git)	lost
box lost	databases restored from the nightly R2 backup; **keys, password and account files are not backed up** and must be set again	lost